INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 2 LECTURE 2

UNIX/LINUX SHELL PROGRAMMING

SHELL SCRIPT

- **Shell Scripts**

- **The basic concept of a shell script is a list of commands, which are listed in the order of execution.**

- **A good shell script will have comments, preceded by # sign, describing the steps.**

- There are conditional tests, such as value A is greater than value B, loops allowing us to go through massive amounts of data, files to read and store data, and variables to read and store data, and the script may include functions.

- **Shell scripts and functions are both interpreted. This means they are not compiled.**

SHELL SCRIPT

- **Example Script**

- Assume we create a **test.sh** script.
- **Note all the scripts would have the .sh extension.**
- Before you add anything else to your script, you need to alert the system that a shell script is being started.
- **This is done using the shebang construct. For example −**

```
#!/bin/bash
```

- This tells the system that the commands that follow are to be executed by the Bourne Again shell. ***It's called a shebang because the # symbol is called a hash, and the ! symbol is called a bang***.
- To create a script containing these commands, you put the shebang line first and then add the commands −

```
#!/bin/bash
pwd
ls
```

SHELL SCRIPT

- **Shell Comments**
- You can put your comments in shell script as follows −

```
#!/bin/bash
 # A simple sample script example
# Store the script  in file test.sh
# Then make the script executable & Run
pwd
ls
```

- Save the above content in test.sh and make the script executable by

```
$chmod +x test.sh
```

- The shell script is now ready to be executed −

```
$./test.sh
```

SHELL SCRIPT

- **Shell Variables**
- **A variable is a character string to which we assign a value. The value assigned could be a number, text, filename, device, or any other type of data.**
- A variable is nothing more than a pointer to the actual data.
- **Variable Names**
- The name of a variable can contain only letters a to z or A to Z , numbers 0 *to* 9 or the underscore character __ .
- By convention, Unix shell variables will have their names in UPPERCASE.
- The following examples are valid variable names −

  _ALI
  TOKEN_A
  VAR_1
  VAR_2

- Following are the examples of invalid variable names −

  2_VAR
  -VARIABLE
  VAR1-VAR2
  VAR_A!

SHELL SCRIPT

- **Defining Variables**
- Variables are defined as follows −

  variable_name=variable_value
- For example −

  GROUP=“JUNIOR CSE ICE"
- The above example defines the variable GROUP and assigns the value " JUNIOR CSE ICE" to it. Variables of this type are called **scalar variables**. A scalar variable can hold only one value at a time.
- Shell enables you to store any value you want in a variable. For example −

  VAR1=“How are you?"

  VAR2=200
- **Accessing Values**
- To access the value stored in a variable, prefix its name with the dollar sign (**$**)
- For example, the following script will access the value of defined variable GROUP and print it on STDOUT −

```
#!/bin/bash
GROUP=“JUNIOR CSE ICE"
echo $GROUP
```

SHELL SCRIPT

WRITING A SHELL SCRIPT - test.sh

```
#!/bin/bash
echo Hi, What is your name
read NAME
echo Good Morning $NAME
echo ============================================================
echo Your Home directory : $PWD
echo Environment variable PATH = $PATH
echo PRESS Enter KEY
read key

echo ============================================================
echo MEMORY FREE SPACE INFO
free
echo PRESS Enter KEY
read key

echo ============================================================
echo PROCESSES CURRENTLY RUNNING
ps -la
echo PRESS Enter KEY
read key
```

SHELL SCRIPT

```
echo ==========================================================
echo LINUX VERSION
uname -a
echo PRESS Enter KEY
read key
 echo ==========================================================
echo WHO ARE CURRENTLY LOGGED IN
who -a
echo PRESS Enter KEY
read key
 echo ==========================================================
echo Do you want directory listing long or short \( enter 1 or 0 \)
read LONG
 if [ $LONG -eq 1 ]
then
ls -la
else
ls
fi
echo PRESS Enter KEY
read key
```

SHELL SCRIPT

```
echo ==========================================================
echo ENTER A FILE NAME TO STORE 10 NAMES
read FILE
echo ENTER 10 NAMES  ONE BELOW THE OTHER ON SEPARAT LINEs and THEN PRESS ^D
cat >> $FILE
echo PRESS Enter KEY
read key
echo ==========================================================
echo your FILE $FILE CONTENTS
cat $FILE
echo PRESS Enter KEY
read key
echo ==========================================================
echo FILE SORTING IN ALPHABETICAL ORDER
sort  $FILE
echo PRESS Enter KEY
read key
echo ==========================================================
echo FILE SORTING IN REVERSE ALPHABETICAL ORDER
sort –r  $FILE
echo PRESS Enter KEY
read key
```

SHELL SCRIPT

```
echo ==========================================
echo Enter a filename to find number of characters, words and lines
read FNAME
cat $FNAME
echo –n No of lines:
wc -l $FNAME
echo –n No of words:
wc -w $FNAME
echo –n No of bytes:
wc -c $FNAME
echo PRESS Enter KEY
read key
```

SHELL SCRIPT

```
echo ==========================================
A=1
B=0
C=1
echo Computing SUM OF FIRST n ODD INTEGERS
echo ENTER VALUE OF n=
read n
while [ $A -le $n ]
do
   B=$(expr $B + $C)
   A=$(expr $A + 1)
   C=$(expr $C + 2)
done
echo Sum of FIRST $n odd numbers = $B
echo PRESS Enter KEY
read key
```

SHELL SCRIPT

```
 echo ==========================================
echo  Files in your HOME Directory with extension .c
for FILE in $HOME/*.c
do
  echo $FILE
done
 echo ==========================================
A=1
B=1
echo Computing FACTORIAL OF n
echo ENTER VALUE OF n=
read n
 until [ $A -gt $n ]
do
   B=$(expr $B \*  $A)
   A=$(expr $A + 1)
done
echo FACTORIAL OF $n = $B
echo PRESS Enter KEY
read key
echo SIGNING OFF ...GOOD BYE
```

SHELL SCRIPT

```
#!/bin/bash
echo COMPUTING SUM OF FIRST 100 INTEGERS
sum=0
for i in {1..100}
do
    sum=$sum+$i
done
echo SUM OF FIRST 100 INTEGERS = $sum
echo ==========================================
```

Create a **shell script copy.sh to provide two command line arguments**

Syntax: $./copy.sh file1 file2

```
#!/bin/bash
echo Copying file $1 to $2
cp $1 $2
echo file $1 CONTENTS
cat $1
echo file $2 CONTENTS
cat $2
```

SHELL SCRIPT

```
#!/bin/bash
read -p  'Login name:' LOGIN
read -sp  'Password:' PASS
echo \n
echo Your LOGIN NAME IS $LOGIN
echo Your PASSWORD IS $PASS


echo  TYPE  NAMES OF YOUR THREE FRIENDS
read FRIEND1 FRIEND2 FRIEND3
echo YOUR FIRST FRIEND IS $FRIEND1
echo YOUR SECOND FRIEND IS $FRIEND2
echo YOUR THIRD FRIEND IS $FRIEND3
echo ========================================
```

SHELL SCRIPT

```
#!/bin/bash
echo ==========================================
echo Name of the Bash script - $0
echo How many arguments were passed to the Bash script - $#
echo All the arguments supplied to the bash script - $@
echo The exit status of the most recently run process  $?
echo The process ID of the current script - $$
echo User Name of the user running the script - $USER
echo The hostname of the machine the script is running on - $HOSTNAME
echo The number of seconds since the script was started - $SECONDS
echo Current line number in the Bash script - $LINENO
echo Random number returned by the RANDOM variable - $RANDOM
echo VALUES OF ALL ENVIRONMENT VARIABLES SET UP IN THE CURRENT ENVIRONMENT
env | more
echo ==========================================
```

BASH SHELL SPECIAL PARAMETERS

SHELL SCRIPT

```
#!/bin/bash
#A simple Menu System
OPTIONS='ls longls who free linuxversion
cat mv cp QUIT'
PS3='Choose an option:'
select CHOICE  in $OPTIONS
do
if [ $CHOICE == ls ]
then
ls
elif [ $CHOICE == longls ]
  then
  ls -la
    elif [ $CHOICE == who ]
    then
    who -a
      elif [ $CHOICE == free ]
      then
      free
      elif [ $CHOICE == linuxversion ]
      then
      uname -a
elif [ $CHOICE == cat ]
   then
    echo ENTER FILE NAME :
        read FILE
        cat $FILE
    elif [ $CHOICE == mv ]
     then
          echo ENTER OLD FILE NAME :
          read FILE1
          echo ENTER NEW FILE NAME :
          read FILE2
          mv $FILE1 $FILE2
     elif [ $CHOICE == cp ]
     then
      echo ENTER NAME OF FILE TO BE COPIED FROM:
      read FILE1
      echo ENTER NAME OF FILE TO BE COPIED TO :
       read FILE2
       cp $FILE1 $FILE2
     elif [ $CHOICE == QUIT ]
      then
         echo BYE ... BYE
         break
      fi
done
```

SHELL SCRIPT

```
#!/bin/bash
loop=1
while [ $loop -eq 1 ]
do
echo ..........................................................................
echo .                        MENU                                          .
echo .                                                                      .
echo .   1. LIST DIRECTORY CONTENTS                                        .
echo .   2. SHOW CURRENT WORKING DIRECTORY                                 .
echo .   3. DISPLAY LINUX VERSION                                          .
echo .   4. SHOW FREE SPACE ON DISK                                        .
echo .   5. SHOW WHO ARE LGGED IN                                          .
echo .   6. DISPLAY CONTENTS OF A FILE                                     .
echo .   7. CREATE OR OPEN A FILE                                          .
echo .   8. COPY A FILE                                                    .
echo .   9. RENAME A FILE                                                  .
echo .   10. REMOVE A FILE                                                 .
echo .   11. QUIT                                                          .
echo ..........................................................................
echo  ENTER YOUR CHOICE :
read CH
```

```
case $CH in
1)
ls
;;
2)
pwd
;;
3)
uname -r
;;
4)
free
;;
5)
who -a
;;
6)
echo Enter File Name :
read FILE
cat $FILE
;;
7)
echo Enter File Name :
read FILE
gedit $FILE
;;
```

SHELL SCRIPT

```
8)
echo Enter Name of the File to be copied from :
read FILE1
echo Enter Name of the File to be copied to :
read FILE2
cp $FILE1 $FILE2
;;
9)
echo Enter OLD File Name:
read FILE1
echo Enter NEW File Name:
read FILE2
mv $FILE1 $FILE2
;;
10)
echo Enter Name of the File to delete:
read FILE
echo ARE YOU SURE - CONFIRM 1 OR 0
read OK
if [ $OK -eq 1 ]
then
rm $FILE
echo File $FILE  deleted.....OK
else
echo File $FILE not deleted.....OK
fi
;;
11)
echo QUITING.....GOOD BYE
break
;;
*)
echo INVALID CHOICE - READ MENU CORRECTLY
;;
esac
done
```

SHELL SCRIPT

- **SHELL FUNCTIONS**

- Functions enable you to break down the overall functionality of a script into smaller, logical subsections, which can then be called upon to perform their individual tasks when needed.

- **Creating Functions**

- **To declare a function, simply use the following syntax −**

```
Function_name()  {
List of commands
}
```

The function name must be followed by parentheses, followed by a list of commands enclosed within braces.

Example :

```
#!/bin/bash
#Define your function here
Hello() {
    echo “Hello World”
}
#Invoke your function
Hello
```

SHELL SCRIPT

Passing Parameters to a Function

- You can define a function that will accept parameters while calling the function. These parameters would be represented by **$1**, **$2** and so on.

Example 1:

```
#!/bin/bash
#Define your function here
Hello() {
  echo "Hello World  $1  $2"
}
#Invoke your function
Hello  GOOD  MORNING
```

Example 2:

```
#!/bin/bash
#Define your function here
Hello() {
  echo "Hello World  $1  $2"
  return 5
}
#Invoke your function
Hello  GOOD  MORNING
 #Capture the value returned by last command
rvalue= $?
echo  "Return value is $rvalue"
```

Nested Functions

One of the more interesting features of functions is that they can call themselves and also other functions

```
func1() {
   echo "currently in  Function 1…"
   echo "Now invoking Function 2…"
func2
}
func2(){
    echo "Currently in Function 2 …."
}
#calling function
func1
```

SHELL SCRIPT

SHELL FUNCTIONS

- Function to compute number of lines, words and bytes in a file
- Shell script name : funfcnt.sh

```
#!/bin/bash
echo ENTER FILENAME
read fname

lines_in_file() {
cat $fname | wc -l
}
words_in_file() {
cat $fname | wc -w
}
bytes_in_file() {
cat $fname | wc -c
}
num_lines=$( lines_in_file )
echo The file $fname has $num_lines lines in it
num_words=$( words_in_file )
echo The file $fname has $num_words words  in it
num_bytes=$( bytes_in_file )
echo The file $fname has $num_bytes bytes in it
```

SHELL SCRIPT

Using Arrays

Example 1:

```
 #!/bin/bash
list=(12 67 123 49 88 123 -9 0 456 126)
let I=0
while [ $I -le 9 ]
do
    echo ${list[$I]}
    let I=$I+1
done
```

Example 2:

```
#!/bin/bash
list=(12 67 123 49 88 123 -9 0 456 126)
let I=0
max=${list[0]}
let I=+$I
while [ $I -le 9 ]
do
    if [ ${list[$I]} -gt $max ]
       then
        max=${list[$I]}
    fi
    let I=$I+1
done
echo Maximum element in the list is : $max
```

Example 3:

```
 #!/bin/bash
echo Enter size of the list :
read N
echo Enter a list of $N numbers :
let K=0
while [ $K -lt $N ]
do
      read VAL
      list[$K]=$VAL
      let K=$K+1
done
let I=0
max=${list[0]}
let I=+$I
while [ $I -lt $N ]
do
    if [ ${list[$I]} -gt $max ]
       then
        max=${list[$I]}
    fi
    let I=$I+1
done
echo Maximum element in the list is : $max
```

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 1 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SHELL PROGRAMMING

Questions

- What is a shell script?
- What is the default shell when you login to linux?
- What is shebang construct. Why is it required?
- Write a Shell script to read a filename from the keyboard and check whether a file exists in your home directory and if it exists then check whether it is a regular file or empty file, directory or symbolic link or block or character device file or named pipe.
- Write a Shell script to read a filename from the keyboard and check whether a file exists in your home directory and if it exists then test whether the file is readable, writeable and executable.
- Write a shell script to read size of the list n and list of n numbers from the keyboard and store the elements in an array list and then find and print the maximum element in the list
- Write a shell script to read size of the list n and list of n numbers from the keyboard and store the elements in an array list and then find and print minimum element in the list
- Write a shell script to read size of the list n and list of n numbers from the keyboard and store the elements in an array list and then search for a given key (key is to be read from the keyboard) in the list – if the given key is found in the list then display the message KEY FOUND and print its position in the list, otherwise print NOT FOUND.

SHELL PROGRAMMING

Questions

- Write a shell script to read a filename from the keyboard and substitute all upper case alphabets with lowercase and store the output in that file.
- Write a shell script to read a filename from the keyboard and substitute all lower case alphabets with uppercase and store the output in that file.
- Write a shell script to read a filename from the keyboard and replace all the occurrences of word “the” in the file with the word “THE” and store the output in that file.
- Write a shell script to read a filename from the keyboard and perform the following operations on the file:
  - First replace all the occurrences of the word “and” with “AND”
  - Next replace all words “the’ in the lines containing the word “AND” to “THE” and store the output in another file
  - Lastly display the contents of both the files.
- Write a shell script to read a filename from the keyboard and perform the following operations on the file :
  - First copy the contents of the file to another file tmp.txt
  - Next delete first and last lines of the file tmp.txt and
  - Lastly display the contents of both the files.

SOC 3010
OPERATING SYSTEM

END OF

WEEK 2 LECTURE 2

SHELL PROGRAMMING

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG